Design de Software - Trabalho Prático 1 - Especificação de Arquitetura

Data de Entrega e Discussão: 27 de Outubro de 2026

1. Enquadramento e objetivos

Em 3 semanas, cada grupo (2–3 elementos) produz uma especificação de arquitetura para o sistema *Lisboa Pedala* e usa-a para gerar, com IA, um esqueleto funcional em Python ou Java. A especificação é o produto principal; o código serve para provar que ela é precisa o suficiente para guiar quem implementa, humano ou modelo.

O trabalho usa apenas matéria das Aulas 1–4: Richards & Ford, *Fundamentals of Software Architecture*, 2.ª ed., caps. 1–5, 8 e 23 e a secção do cap. 9 sobre particionamento de topo, e ainda as estruturas e vistas da Aula 4 (Bass, Clements & Kazman; *Views and Beyond*). Ficam de fora o cap. 6 (medir e governar), o cap. 7 (*architecture quantum*) e o resto do cap. 9. No fim, cada aluno deve ser capaz de:

1. Identificar características arquiteturais explícitas e implícitas a partir de requisitos e contexto, e escolher as 3 que conduzem a arquitetura (caps. 4–5, Aula 3).
2. Distinguir arquitetura lógica de física e construir componentes lógicos, atribuindo-lhes *user stories* sem cair na *entity trap* (cap. 8, Aula 4).
3. Analisar coesão e acoplamento entre componentes: aferente, eferente, instabilidade, conascência, acoplamento temporal e Lei de Deméter (caps. 3 e 8, Aulas 2 e 4).
4. Escolher o particionamento de topo, técnico ou por domínio, com o *trade-off* explícito (cap. 9, só esta secção).
5. Descrever o sistema com vistas de módulos, de componentes e conectores e de alocação, incluindo regras *allowed-to-use* (Aula 4).
6. Registar decisões em ADRs e comunicar a arquitetura com diagramas C4 consistentes (cap. 23, Aula 4).
7. Escrever uma especificação que uma ferramenta de IA consiga seguir, e verificar automaticamente as regras de dependências com *fitness functions* (Aula 4).

2. O sistema: kata *Lisboa Pedala*

O enunciado segue o formato de *architecture kata* (Neward): descrição, utilizadores, requisitos e contexto adicional. O sistema é fictício. Muitas das características importantes não estão escritas; descobri-las faz parte do trabalho.

Descrição. Uma empresa municipal vai substituir o sistema de bicicletas partilhadas de Lisboa. Precisa de uma plataforma nova para desbloquear e devolver bicicletas, cobrar viagens, manter a frota e reportar à Câmara.

Utilizadores.

Perfil	Volume
Assinantes (passe anual)	~60 000
Utilizadores ocasionais	~15 000 por mês
Técnicos de manutenção	25
Equipas de redistribuição (carrinhas)	6
Gestores municipais	~10

Requisitos.

1. Desbloquear uma bicicleta na estação com a app (código QR) ou com o cartão Navegante.
2. Devolver em qualquer estação com doca livre; a viagem fecha quando a doca confirma o encaixe.
3. Tarifário: o passe anual inclui 45 min grátis por viagem, depois cobra-se por bloco de 15 min; os ocasionais pagam por viagem (MB WAY ou cartão).
4. Mapa em tempo real com bicicletas e docas livres por estação.
5. Abrir uma ordem de manutenção quando uma bicicleta reporta avaria (bateria, travões) ou um utilizador a reporta.
6. Sugerir rotas às carrinhas para equilibrar a ocupação das estações.
7. Relatório mensal para a Câmara: utilização por freguesia e CO_2 evitado.
8. O utilizador consulta o histórico de viagens e as faturas.

Contexto adicional.

- Frota de 1 500 bicicletas (70% elétricas) em 180 estações. As elétricas enviam telemetria a cada 30 s em andamento.
- Cerca de 70% dos desbloqueios acontecem entre as 8h00–9h30 e as 17h30–19h00. Em eventos (Santos Populares, maratona) a procura em algumas estações multiplica por 4.
- As docas comunicam por rede móvel e há zonas onde a rede falha com frequência. Cada doca guarda até 24 h de eventos localmente.
- O contrato de concessão aplica uma multa por cada hora em que mais de 10% das estações não conseguem desbloquear bicicletas.
- As faturas são comunicadas à Autoridade Tributária e guardadas durante 10 anos. O percurso de cada viagem é dado pessoal (RGPD).
- Dentro de um ano, a Câmara quer publicar a ocupação das estações como dados abertos.
- Duas câmaras vizinhas mostraram interesse em usar a plataforma com tarifários próprios.
- Equipa de desenvolvimento de 6 pessoas e orçamento fixo.

3. Entregáveis

Entrega-se um único repositório git com a especificação em Markdown, os diagramas e o código. O histórico de *commits* faz parte da avaliação: tem de mostrar a especificação a nascer antes do código.

```
lisboa-pedala/
├── ARCHITECTURE.md          # especificação para IA (ver 3.2)
├── docs/
│   ├── 01-caracteristicas.md
│   ├── 02-componentes.md
│   ├── 03-particionamento-e-vistas.md
│   ├── adr/                 # ADR-001.md, ADR-002.md, ...
│   └── diagramas/           # Mermaid, PlantUML ou Structurizr DSL
├── src/
├── tests/
│   ├── architecture/        # fitness functions
│   └── ...
├── CODEOWNERS               # vista de work assignment
└── prompts/                 # registo de uso de IA (secção 4)
```

3.1 Especificação de arquitetura

Ficheiro	Matéria	Conteúdo mínimo
`01-caracteristicas.md`	Caps. 4–5, Aula 3	*Worksheet*: até 7 candidatas, implícitas em coluna à parte, top 3 sem ordenar, "outras consideradas". Cada uma ligada à frase do enunciado de onde saiu. Pelo menos uma característica composta decomposta. Para cada uma das top 3, um cenário concreto do que significa neste sistema.
`02-componentes.md`	Caps. 3 e 8, Aulas 2 e 4	Componentes lógicos (abordagem *workflow* ou *actor/action*, dizer qual), mapeamento *user story* → componente, análise de coesão, acoplamento aferente e eferente e instabilidade por componente, conascência entre pelo menos 2 componentes, 2 exemplos de acoplamento temporal. Uma iteração de reestruturação documentada, com o motivo. Evitar a *entity trap*.
`03-particionamento-e-vistas.md`	Cap. 9 (particionamento), Aula 4	Particionamento de topo, técnico ou por domínio, com o *trade-off*. Vista de módulos com as regras *allowed-to-use*. Vista de componentes e conectores do fluxo desbloquear→devolver→cobrar. Vista de

		alocação: onde corre cada parte e que elemento do grupo é responsável por cada componente (`CODEOWNERS`).
`adr/`	Aula 4	Mínimo 5 ADRs (contexto, decisão, consequências, alternativas rejeitadas). Pelo menos 1 tem de ser revisto ou substituído durante o trabalho.
`diagramas/`	Cap. 23, Aula 4	C4 níveis 1–3 (contexto, contentores, componentes), coerentes com as vistas do ficheiro 03. Notação consistente e legenda em todos.

3.2 `ARCHITECTURE.md` : a especificação para IA

É o ficheiro que a ferramenta de IA lê antes de gerar código (pode ser referenciado por `AGENTS.md` , `CLAUDE.md` ou equivalente). Máximo de 4 páginas. Tem de conter:

- Lista de componentes, a responsabilidade de cada um numa frase e o *namespace* ou pacote onde vive.
- Regras *allowed-to-use* entre componentes, escritas como regras verificáveis e não como sugestões.
- As top 3 características e o que cada uma implica para o código.
- O acoplamento temporal a respeitar: ordem das operações e o que tem de acontecer numa só unidade de trabalho.
- Convenções de código e uma lista curta do que a IA não pode fazer.
- Referência aos ADRs relevantes por componente.

3.3 Código

- Python 3.11+ ou Java 21, à escolha do grupo.
- Um esqueleto funcional com todos os componentes lógicos criados, mesmo que vazios, organizados segundo o particionamento de topo escolhido.
- 3 *user stories* implementadas de ponta a ponta. A US-1 é obrigatória; as outras duas escolhem-se de entre as restantes:
  - **US-1** (obrigatória): como assinante, desbloqueio e devolvo uma bicicleta e a viagem fica registada e cobrada.
  - **US-2**: como técnico, recebo uma ordem de manutenção quando uma bicicleta reporta avaria.
  - **US-3**: como doca sem rede, reenvio os eventos guardados quando a rede volta, sem duplicar viagens.
  - **US-4**: como gestor municipal, obtenho o relatório de utilização por freguesia.
- Pelo menos uma das *user stories* tem de envolver 3 ou mais componentes.

- Interface por CLI ou API REST; não é pedida interface gráfica. Persistência em SQLite ou em memória. Docas e pagamentos são simulados.
- *Fitness functions* em `tests/architecture/`, a correr num único comando (ou em CI): verificam as regras *allowed-to-use* da vista de módulos e a ausência de ciclos entre componentes. Ferramentas dadas na Aula 4: `import-linter` (Python) ou ArchUnit (Java).

3.4 Relatório de reflexão

Máximo de 2 páginas em `docs/reflexao.md`: o que a especificação não previu, que violações a IA cometeu, quais as *fitness functions* apanharam e quais passaram despercebidas.

4. Uso de IA

A IA pode gerar todo o código e os testes, e pode ser usada para criticar a especificação. Não há penalização por usar IA; há penalização por não a declarar e por não saber defender o que ela produziu.

Regras.

1. Qualquer ferramenta é permitida. Indicar no `README.md` quais foram usadas e com que modelo.
2. O código é gerado **a partir da especificação**: cada pedido de geração de um componente tem de referir `ARCHITECTURE.md` e o ADR relevante.
3. A análise de características, os *componentes lógicos* e as decisões dos ADRs são do grupo. A IA pode servir de revisor crítico, mas a escolha e a justificação têm de ser defendíveis por cada elemento sem ajuda.
4. Nenhum código entra no repositório sem passar nas *fitness functions*.

Registo em `prompts/`. Um ficheiro por componente gerado, com:

- o pedido feito e os ficheiros de contexto fornecidos;
- o resultado aceite ou rejeitado, e porquê;
- se houve violação da arquitetura, quem a detetou: a *fitness function*, a revisão humana, ou ninguém até mais tarde.

São exigidos pelo menos 3 casos documentados em que o código gerado violou a especificação. Se o grupo não encontrar nenhum, a explicação mais provável é que as *fitness functions* são fracas, não que a IA acertou sempre.

Teste de regeneração (em aula, semana 3). O docente escolhe um componente. O grupo apaga-o e regenera-o com IA usando apenas o repositório e `ARCHITECTURE.md`, em 30 minutos e até 2 tentativas. O resultado passa se as *fitness functions* e os testes das *user stories* continuarem verdes. Este teste mede a especificação, não a IA: uma especificação vaga produz um componente diferente de cada vez.

5. Calendário

O trabalho dura 3 semanas, com dois checkpoints nas aulas práticas do fim das semanas 1 e 2 e a entrega no fim da semana 3. Os checkpoints não contam para a nota: servem para corrigir a especificação antes de o código depender dela.

6. Critérios de avaliação

A nota de grupo vale 20 valores e pesa mais a especificação do que o código: 65% contra 35%. Não há resposta "certa" para a arquitetura; avalia-se se cada escolha está ligada a uma característica, a um *trade-off* e a uma verificação.

7. Avaliação individual em aula

Cada aluno faz uma discussão individual de 10 minutos sobre o repositório do grupo, sem acesso a ferramentas de IA.